

TypeScript 零基础入门教程

一、什么是TypeScript?

TypeScript (简称TS) 是JavaScript的超集, 它在JS的基础上增加了**静态类型系统**, 可以在代码编写阶段就发现类型错误, 而不是运行时。

- 类比理解: JS像“草稿纸”, 写的时候不检查格式; TS像“正式文档”, 写的时候就要求格式规范, 最后还能编译成标准JS运行。
- 核心优势: 代码可维护性更高、编辑器智能提示更友好、大型项目协作更高效。

二、环境准备

1. 安装Node.js

TS最终要编译成JS运行, 需要Node.js环境:

- 下载地址: <https://nodejs.org/> (推荐LTS长期支持版)
- 验证安装: 终端执行 `node -v` 和 `npm -v` 能显示版本号即可。

2. 安装TypeScript

打开终端, 执行全局安装命令:

```
npm install -g typescript
```

验证安装: 执行 `tsc -v` 显示版本号 (如v5.3.3) 则安装成功。

三、第一个TS程序

1. 编写TS代码

新建文件 `hello.ts`, 写入以下代码:

```
// 定义一个带类型的变量
const message: string = "Hello TypeScript!";
// 定义一个带参数类型和返回值类型的函数
function sayHello(name: string): string {
    return `Hello, ${name}!`;
}

// 调用函数并打印
console.log(message);
console.log(sayHello("TS新手"));
```

- 关键说明: `string` 是TS的类型注解, 指定变量/参数/返回值必须是字符串类型。

2. 编译TS为JS

终端执行编译命令:

```
tsc hello.ts
```

执行后会生成 `hello.js` 文件（TS编译后的JS代码）。

3. 运行JS代码

```
node hello.js
```

输出结果：

```
Hello TypeScript!  
Hello, TS新手!
```

四、核心语法（入门必学）

1. 基础类型

TS提供了JS没有的明确类型注解，常用基础类型如下：

```
// 字符串  
let name: string = "张三";  
// 数字（整数/浮点数都适用）  
let age: number = 20;  
// 布尔值  
let isStudent: boolean = true;  
// 数组（两种写法）  
let arr1: number[] = [1, 2, 3];  
let arr2: Array<string> = ["a", "b", "c"];  
// 空值（无返回值的函数）  
function doNothing(): void {  
    console.log("无返回值");  
}  
// 任意值（关闭TS类型检查，慎用）  
let anyValue: any = "任意类型";  
anyValue = 123; // 不会报错
```

2. 接口 (Interface)

用于约束对象的结构，是TS最核心的特性之一：

```
// 定义接口  
interface Person {  
    name: string; // 必选属性  
    age: number;  
    gender?: string; // 可选属性（加?）  
    readonly id: number; // 只读属性（不可修改）  
}  
  
// 使用接口约束对象  
const person1: Person = {  
    name: "李四",  
    age: 25,  
    id: 1001 // 只读，后续不能修改  
};  
  
// 错误示例：缺少age属性会报错  
// const person2: Person = { name: "王五" };
```

```
// 错误示例：修改只读属性会报错
// person1.id = 1002;
```

3. 函数类型

约束函数的参数类型和返回值类型：

```
// 方式1：直接注解
function add(a: number, b: number): number {
    return a + b;
}

// 方式2：类型别名（复用函数类型）
type AddFunc = (x: number, y: number) => number;
const add2: AddFunc = (a, b) => a + b;

// 可选参数（必须放在必选参数后面）
function printInfo(name: string, age?: number): void {
    console.log(`姓名: ${name}, 年龄: ${age || "未知"}`);
}

// 默认参数
function greet(name: string = "游客"): string {
    return `你好, ${name}!`;
}
```

4. 类 (Class) 与面向对象

TS增强了JS的类语法，支持访问修饰符、接口实现等：

```
// 定义类
class Animal {
    // 私有属性（仅类内部可访问）
    private name: string;
    // 受保护属性（类内部+子类可访问）
    protected age: number;
    // 公共属性（默认public，外部可访问）
    public type: string;

    // 构造函数
    constructor(name: string, age: number, type: string) {
        this.name = name;
        this.age = age;
        this.type = type;
    }

    // 方法
    say(): void {
        console.log(`我是${this.type}, 叫${this.name}, ${this.age}岁`);
    }
}

// 子类继承
class Dog extends Animal {
    constructor(name: string, age: number) {
        super(name, age, "狗"); // 调用父类构造函数
    }
}
```

```

    }

    // 重写父类方法
    say(): void {
        console.log(`汪汪! 我是${this.type}, 叫${this.name}`);
    }
}

// 创建实例
const dog = new Dog("旺财", 3);
dog.say(); // 输出: 汪汪! 我是狗, 叫旺财

```

5. 泛型 (Generic)

解决“复用代码但不丢失类型信息”的问题，是TS的高级特性：

```

// 泛型函数: T是类型参数, 代表任意类型
function identity<T>(arg: T): T {
    return arg;
}

// 使用时指定类型
const str = identity<string>("hello"); // str类型为string
const num = identity<number>(123); // num类型为number

// 泛型接口
interface Result<T> {
    success: boolean;
    data: T;
}

// 泛型类
class Box<T> {
    private content: T;
    constructor(content: T) {
        this.content = content;
    }
    getContent(): T {
        return this.content;
    }
}

```

五、TS配置文件 (tsconfig.json)

实际开发中，我们不会手动编译单个文件，而是通过 `tsconfig.json` 配置编译规则：

1. 生成配置文件

```
tsc --init
```

2. 核心配置项说明

```
{
  "compilerOptions": {
    "target": "ES6", // 编译后的JS版本（ES5/ES6/ESNext）
    "module": "ESNext", // 模块系统（CommonJS/ESModule）
    "outDir": "./dist", // 编译后的文件输出目录
    "rootDir": "./src", // 源代码目录
    "strict": true, // 开启严格类型检查（推荐开启）
    "esModuleInterop": true // 兼容CommonJS和ESModule
  },
  "include": ["./src/**/*"], // 要编译的文件
  "exclude": ["node_modules"] // 排除的文件
}
```

3. 批量编译

配置完成后，终端执行 `tsc` 即可自动编译 `src` 下所有TS文件到 `dist` 目录。

总结

1. TypeScript是JS的超集，核心优势是**静态类型检查**，能提前发现错误，提升代码可维护性。
2. 入门必掌握：基础类型、接口、函数类型、类、泛型，这些是TS最常用的核心语法。
3. 实际开发中通过 `tsconfig.json` 配置编译规则，配合编辑器（VSCode）的智能提示，能大幅提升开发效率。

如果想深入学习，可以先从“类型注解+接口”入手，结合小案例（如封装一个工具函数、定义一个用户对象）练习，再逐步学习泛型、高级类型等进阶内容。